

Kalman Filtering II: The Steady-State Kalman Filter

The gain the recursion converges to – computed directly.

In `Intro.m` the Kalman gain settled to a constant. For a time-invariant system with stationary noise you can compute that **steady-state** gain without iterating and run the filter with a fixed gain – i.e. an optimal Luenberger observer. In this tutorial you will:

- get the steady-state gain from the filter Riccati equation with `idare`,
- confirm it equals the fixed point of the covariance recursion, and
- show it beats any other fixed observer gain.

Reuses the constant-velocity model and connects to `State-Space/StateObservers.m`. Run with `publish('SteadyStateKalman.m')`.

Contents

- Model and noise (same constant-velocity target as Intro)
- Steady-state gain from the filter Riccati equation
- ... and as the fixed point of the covariance recursion
- The optimal fixed gain beats any other fixed gain
- Try it yourself

Model and noise (same constant-velocity target as Intro)

```
dt = 0.1;
A = [1 dt; 0 1];
C = [1 0];
G = [dt^2/2; dt];           % process-noise (acceleration) input
q = 0.2;                    % process-noise variance
R = 4;                      % measurement-noise variance
```

Steady-state gain from the filter Riccati equation

The predicted covariance converges to the solution of a discrete algebraic Riccati equation, which `idare` solves directly (the **dual** of the control DARE). The toolbox one-liners `dlqe` and `kalman` wrap this same solve.

```
Pp = idare(A', C', G*q*G', R);
K_ss = Pp*C' / (C*Pp*C' + R);
fprintf('Steady-state Kalman gain (idare):    [%.4f; %.4f]\n', K_ss)
```

```
Steady-state Kalman gain (idare):    [0.0647; 0.0216]
```

... and as the fixed point of the covariance recursion

The steady gain is simply where the predict/update recursion stops changing. Iterate it and confirm it lands on the same gain.

```
P = eye(2);
for k = 1:500
    P = A*P*A' + G*q*G';           % predict
    K = P*C'/(C*P*C' + R);         % gain
    P = (eye(2) - K*C)*P;          % update
end
fprintf('Steady-state Kalman gain (iterated): [%.4f; %.4f]\n', K)
fprintf('Difference: %.2e\n', norm(K - K_ss))
```

```
Steady-state Kalman gain (iterated): [0.0647; 0.0216]
Difference: 6.49e-16
```

The optimal fixed gain beats any other fixed gain

A Luenberger observer runs with **any** stabilizing fixed gain; the steady-state Kalman filter is the fixed gain that **minimizes the estimation-error variance**. Compare it against a timid (low) gain and an aggressive (high) gain on the same data.

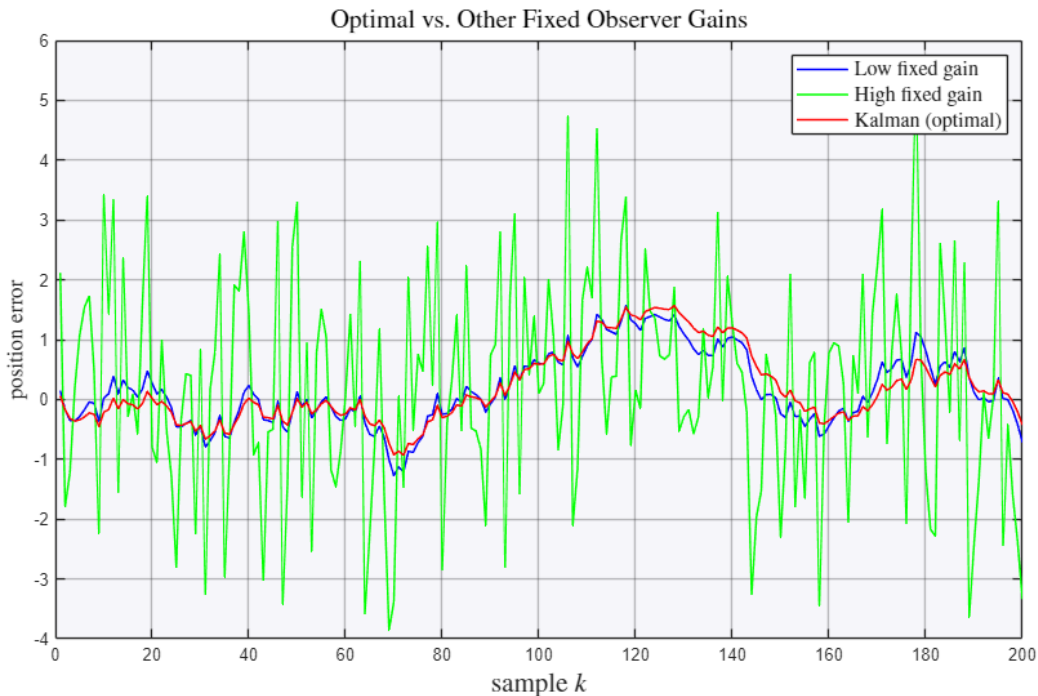
```
rng(1); N = 200;
x = [0;1]; X = zeros(2,N); Y = zeros(1,N);
for k = 1:N
    x = A*x + G*sqrt(q)*randn;
    X(:,k) = x; Y(k) = C*x + sqrt(R)*randn;
end

gains = {K_ss, [0.10;0.02], [0.90;0.25]}; % optimal, low, high
names = {'Kalman (optimal)', 'Low fixed gain', 'High fixed gain'};
errs = zeros(3,N);
for g = 1:3
    Kg = gains{g}; xh = [0;0];
    for k = 1:N
        xh = A*xh; % predict
        xh = xh + Kg*(Y(k) - C*xh); % update with a FIXED gain
        errs(g,k) = xh(1) - X(1,k);
    end
end

figure
plot(1:N, errs(2,:), 'b', 1:N, errs(3,:), 'g', 1:N, errs(1,:), 'r', 'LineWidth', 1.1)
grid on
legend(names{2}, names{3}, names{1}, 'Interpreter', 'latex', 'FontSize', 12)
title('Optimal vs. Other Fixed Observer Gains', 'Interpreter', 'latex', 'FontSize', 15)
ylabel('position error', 'Interpreter', 'latex', 'FontSize', 13)
xlabel('sample $k$', 'Interpreter', 'latex', 'FontSize', 16)

for g = 1:3
    fprintf('RMS error -- %-18s %.3f\n', [names{g} ' ':'], sqrt(mean(errs(g,:).^2)))
end

RMS error -- Kalman (optimal): 0.637
RMS error -- Low fixed gain: 0.630
RMS error -- High fixed gain: 1.825
```



Try it yourself

- Try any fixed gains you like and notice you can never beat the Kalman RMS error – optimality is a hard lower bound.
- Change $\mathbf{R}$ and re-run: `idare` and the iteration track the new steady gain and keep agreeing.